WAYSTAR

WAYSTAR HACKATHON CHALLENGE

# Quick Payment Pages (QPP)

Participant Guide & Technical Requirements

*Build a self-service, configurable online payment experience*

| Full-Stack | Payment Processing | WCAG Accessible |
| --- | --- | --- |
| Web Application | & Integration | UI/UX Design |

# 1. Challenge Overview

Welcome to the Quick Payment Pages (QPP) Hackathon Challenge! Your mission is to design and build a hosted, full-stack web application that enables providers to create flexible, branded, self-service payment pages. This challenge simulates a real-world product build and will test your skills across frontend, backend, payment integration, accessibility, and reporting.

> **Objective:** Build a working QPP platform where administrators can configure payment pages, and end users can make payments — all served through a publicly accessible hosted URL.

## 1.1 What is QPP?

Quick Payment Pages (QPPs) are long-lived, self-service online payment experiences. Unlike single-use checkout flows, QPPs are persistent pages that can be reused over time for recurring service payments. Examples include:

- Yoga studio class sign-ups and recurring membership fees
- Municipal parking fee payments
- Utility or miscellaneous service payments
- Event registrations and donations

## 1.2 Who Are the Users?

| User Type | Description | Key Actions |
|---|---|---|
| Provider Admin | Staff at the service organization (e.g., yoga studio manager) | Create & configure payment pages, view reports |
| End User / Payer | Customer making a payment for a service | Visit payment page, fill in fields, submit payment |
| System | Automated processes | Route payments, send confirmations, log transactions |

# 2. Functional Requirements

Your application must implement the following functional areas. Each is described with enough detail to guide your implementation decisions.

## 2.1 Payment Page Configuration (Admin Portal)

Provider administrators must be able to log in to an admin portal and perform the following actions:

### 2.1.1 Create & Manage Payment Pages

- Create new Quick Payment Pages with a unique URL slug (e.g., /pay/yoga-class)
- Edit existing page configurations
- Enable or disable a payment page without deleting it
- View a list of all configured payment pages with status indicators

### 2.1.2 Branding & Styling

- Upload or specify an organization logo
- Set a primary brand color (hex code or color picker)
- Define a page title and subtitle/description
- Set a custom header and footer message
- Preview the payment page in real-time as configuration is applied

### 2.1.3 Payment Amount Configuration

Administrators must be able to configure one of three payment amount modes per page:

| Mode | Description | Example |
|---|---|---|
| Fixed Amount | A pre-set, non-editable amount that every payer must pay | $25.00 class fee |
| Min/Max Range | Payer enters an amount within a defined range | Between $10 and $500 |
| User-Entered | Payer enters any amount they choose (e.g., donations) | Open amount entry |

### 2.1.4 Custom Data Fields

Administrators must be able to add up to 10 custom input fields to collect additional information from the payer. Each field must support:

- Field label / display name
- Field type: Text, Number, Dropdown (with configurable options), Date, Checkbox
- Required vs. optional designation
- Placeholder text or helper text
- Display order (drag-and-drop or orderable)

> **Example:** A yoga studio might add: Student Name (text, required), Class Date (date, required), Membership Type (dropdown: Monthly / Annual, required), Special Requests (text, optional).

#### 2.1.5 General Ledger (GL) Codes

- Administrators must be able to associate one or more GL codes with each payment page
- GL code must be stored and associated with each completed transaction for reporting purposes
- UI must allow entry and validation of GL code format

#### 2.1.6 Email Confirmation Templates

- Administrators can define a custom confirmation email template per payment page
- Template must support dynamic variables: payer name, amount paid, transaction ID, date, custom field values
- A default template must be used if no custom template is defined
- Confirmation emails must be sent automatically upon successful payment

---

### 2.2 Link Distribution

QPPs must support the following sharing and distribution channels:

#### 2.2.1 Direct URL

- Every QPP must have a unique, human-readable public URL (e.g., https://yourdomain.com/pay/yoga-class)
- URL must be copyable from the admin portal with one click
- URL must be shareable via email, SMS, or any messaging platform

#### 2.2.2 Embeddable Iframe

- Admin portal must generate an embeddable HTML iframe snippet for each QPP
- Snippet must be copyable and renderable on any external website
- Iframe must be responsive and mobile-friendly

#### 2.2.3 QR Code

- Admin portal must generate a downloadable QR code for each QPP
- QR code must resolve to the QPP's public URL when scanned
- QR code must be downloadable as PNG or SVG

## 2.3 Payment Methods

The QPP should support the following online payment methods. You may use a sandbox/test environment for a payment processor of your choice (e.g., Stripe, Braintree, Square).

### 2.3.1 Basic Payment

- Implement a checkout form that captures standard payment details, specifically the card number, expiration date, CVV, and billing zip code.

### 2.3.2 Credit Card (Stretch Goal)

- Accept Visa, Mastercard, and American Express
- Collect card number, expiration date, CVV, and billing zip code
- Implement real-time card validation (Luhn algorithm or processor validation)
- Display clear error messages for declined or invalid cards

### 2.3.3 Digital Wallet (Stretch Goal)

- Support at least one digital wallet method (e.g., Apple Pay, Google Pay, or PayPal)
- Wallet availability should be detected dynamically based on user device/browser

### 2.3.4 ACH / Bank Transfer (Stretch Goal)

- Accept bank account number and routing number
- Display ACH authorization language required by NACHA rules
- Clearly communicate processing time (2-3 business days)

**Note:** All payment processing must be implemented using a sandbox/test mode. No real financial transactions are expected. Document which payment processor you chose and why.

## 2.4 Accessibility (WCAG Compliance)

All public-facing payment pages must meet WCAG 2.1 Level AA accessibility standards. This is a mandatory requirement, not optional.

- All form inputs must have properly associated labels

- Color contrast ratio must meet 4.5:1 for normal text and 3:1 for large text
- All interactive elements must be keyboard navigable (Tab, Enter, Space, Arrow keys)
- All images and icons must have appropriate alt text
- Error messages must be programmatically associated with their fields (aria-describedby)
- Page must be screen-reader compatible
- Focus indicators must be clearly visible on all interactive elements
- No content may rely solely on color to convey meaning

**Tip:** Use automated tools like axe DevTools, Lighthouse (in Chrome DevTools), or WAVE during development to catch accessibility issues early.

## 2.5 Reporting

The admin portal must include a reporting section that allows administrators to view and export payment activity.

### 2.5.1 Required Report Views

- Transaction list with filters: date range, payment page, status (success/failed/pending)
- Summary view: total payments, total amount collected, average payment amount
- Breakdown by GL code
- Breakdown by payment method (credit card vs. wallet vs. ACH)

### 2.5.2 Export (Stretch Goal)

- Reports must be exportable to CSV
- Export must respect active filters (export what you see)

## 2.6 Product Differentiator

Create a feature for QPP that you believe would deliver clear value to providers and payers, and that would meaningfully differentiate your solution from what other teams build (or what a typical “basic payment link” experience offers). Be creative—think about real-world needs like trust and transparency, a smoother payer experience, smarter admin configuration, better reporting insights, or easier distribution—and be prepared to explain why your differentiator matters, who it helps, and how it would work at a high level.

# 3. Technical Requirements

## 3.1 Application Architecture

Your application must be a hosted, full-stack web application accessible via a public URL. The following architectural requirements apply:

| Layer | Requirements |
|---|---|
| Frontend | Modern web framework (React, Vue, Angular, or Svelte). Responsive design. No jQuery-only implementations. |
| Backend / API | RESTful or GraphQL API. Any server-side language (Node.js, Python, Java, Go, etc.). API must be documented (README or Swagger). |
| Database | Any relational or NoSQL database. Schema must be included in submission. |
| Hosting | Must be publicly accessible via HTTPS URL. Use any cloud provider (AWS, GCP, Azure, Heroku, Railway, Vercel, Render, etc.). |
| Authentication | Admin portal must require login. Basic auth, JWT, or OAuth all acceptable. |

## 3.2 Technology Stack Guidance

You are free to choose your own technology stack. The following are suggestions, not requirements:

| Component | Suggested Options | Notes |
|---|---|---|
| Frontend | React + Vite, Next.js, Vue 3 | Next.js simplifies hosting on Vercel |
| Backend | Node/Express, FastAPI, Spring Boot | Pick what you know best |
| Database | PostgreSQL, MySQL, MongoDB, SQLite | Hosted: Supabase, PlanetScale, MongoDB Atlas |
| Payments | Stripe (recommended), Square, Braintree | Stripe has the best sandbox tooling |
| QR Code | qrcode.js, qrcode (npm), python-qrcode | Client-side generation is fine |
| Auth | Auth0, Clerk, Supabase Auth, custom JWT | Avoid building auth from scratch |
| Hosting | Vercel, Railway, Render, AWS, GCP | Free tiers available on Vercel/Railway |

## 3.3 Security Requirements

- All payment page URLs must use HTTPS
- Do not use real card data.
- Admin portal must be protected by authentication
- Environment variables must be used for all secrets and API keys (no hardcoded credentials)
- Input validation must be performed on both client and server sides

# 4. Deliverables

At the end of the hackathon, each participant or team must submit the following:

## 4.1 Working Application

- Publicly accessible hosted URL (HTTPS)
- Admin portal with login credentials provided to judges
- At least two pre-configured demo payment pages
- At least one completed test transaction visible in the reporting section

## 4.2 Source Code

- Public or shared GitHub/GitLab repository
- README.md with: project description, setup instructions, environment variable guide, and architecture diagram or description
- Database schema (SQL file, migrations, or entity diagram)

## 4.3 Demo Presentation (5-10 Minutes)

1. Walk through the admin portal — create or edit a payment page
2. Demonstrate link distribution — show the URL, iframe snippet, and QR code
3. Demonstrate mock payment functionality
4. Show the transaction in the reporting section
5. Briefly explain your technology choices and any notable challenges

# 5. Judging Criteria

Projects will be evaluated across five dimensions. Each dimension is weighted as shown below:

| Criteria | Weight | What Judges Will Look For |
|---|---|---|
| Functionality & Completeness | 30% | Does the app meet the core requirements? Are all major features implemented and working? |
| User Experience & Design | 20% | Is the UI intuitive and professional? Is the payment flow smooth and trustworthy? |
| Accessibility (WCAG) | 10% | Does the payment page pass automated accessibility checks? Is it keyboard navigable? |
| Code Quality & Architecture | 20% | Is the code clean, organized, and well-documented? Are security best practices followed? |
| Demo & Communication | 20% | Is the demo well-prepared? Can the participant explain their decisions clearly? |

## 6. Getting Started — Recommended Approach

If you are unsure where to begin, the following phased approach is recommended. Adjust based on your experience level and preferred stack.

### Phase 1: Foundation (First 2 Hours)

6. Set up your project repository and hosting environment
7. Initialize your frontend and backend scaffolding
8. Set up your database and define your core schema (pages, fields, transactions)
9. Implement basic admin authentication (login/logout)

### Phase 2: Core Features (Hours 3–6)

10. Build the payment page configuration UI in the admin portal
11. Implement the public-facing payment page renderer (reads config, renders form)
12. Integrate your chosen payment processor in sandbox/test mode
13. Implement confirmation email sending (use Sendgrid, Resend, or Nodemailer)

### Phase 3: Distribution & Reporting (Hours 7–9)

1. Add URL copy, iframe snippet generation, and QR code download to the admin portal
2. Build the reporting page with transaction list, filters, summary stats, and CSV export

### Phase 4: Polish & Accessibility (Final Hour)

1. Run Lighthouse accessibility audit and fix flagged issues
2. Verify keyboard navigation and focus management on the payment page
3. Finalize README and prepare demo script

## 7. Suggested Data Model

The following is a suggested starting point for your database schema. You are not required to follow this exactly, but your schema must support all functional requirements.

### Core Entities

| Entity | Key Fields | Description |
|---|---|---|
| PaymentPage | id, slug, title, description, brand_color, logo_url, amount_mode, fixed_amount, min_amount, max_amount, gl_code, is_active | The core QPP configuration |
| CustomField | id, page_id, label, type, options, required, order | Custom fields attached to a page |
| Transaction | id, page_id, amount, payment_method, status, payer_email, created_at | Payment record for each submission |
| FieldResponse | id, transaction_id, field_id, value | Custom field values per transaction |
| AdminUser | id, email, password_hash, created_at | Admin portal users |

## 8. Hackathon Rules & Guidelines

### 8.1 General Rules

- You may work individually or in teams of up to 3 people
- All code must be written during the hackathon — pre-built templates or existing projects are not allowed
- Use of open-source libraries, UI component libraries, and AI coding assistants is permitted
- You may use any programming language, framework, or cloud provider
- All submitted applications must be hosted and publicly accessible at demo time

### 8.2 What Is and Is Not Allowed

| Allowed | Not Allowed |
|---|---|
| Open-source libraries and npm/pip packages | Pre-built commercial templates or SaaS page builders |
| AI coding assistants (GitHub Copilot, Claude, ChatGPT) | Submitting a project started before the hackathon |
| Payment processor sandbox/test environments | Real payment processing or live transactions |
| UI component libraries (MUI, Tailwind, shadcn) | Hardcoded credentials or secrets in source code |
| Cloud free tiers for hosting and databases | Plagiarism or using another participant's code |

### 8.3 Stretch Goals (Bonus Points)

The following features are not required but will earn bonus consideration during judging:

- ACH / bank transfer payment method implementation
- Multi-language / internationalization (i18n) support
- Webhook support for payment processor events (e.g., failed charges, refunds)
- Dark mode support with user preference detection
- End-to-end tests (Playwright or Cypress)
- Mobile app wrapper using React Native or Flutter

## 9. Helpful Resources

### Payment Processing

- Stripe Docs: https://stripe.com/docs
- Stripe Test Cards: https://stripe.com/docs/testing

- Stripe Elements (prebuilt UI): https://stripe.com/docs/payments/elements

### Accessibility

- WCAG 2.1 Guidelines: https://www.w3.org/TR/WCAG21/
- axe DevTools Browser Extension: https://www.deque.com/axe/
- WebAIM Contrast Checker: https://webaim.org/resources/contrastchecker/

### QR Code Libraries

- qrcode (npm): https://www.npmjs.com/package/qrcode
- react-qr-code: https://www.npmjs.com/package/react-qr-code

### Free Hosting Platforms

- Vercel (ideal for Next.js / React): https://vercel.com
- Railway (backend + database): https://railway.app
- Render (full-stack): https://render.com
- Supabase (PostgreSQL + Auth): https://supabase.com

## 10. Submission Checklist

Use this checklist to verify your submission is complete before demo time:

> **Reminder:** A partially complete application that works well in the implemented areas is better than a broken application that claims full coverage. Focus on quality over quantity.

### Application

- Hosted application accessible via public HTTPS URL
- Admin portal protected by authentication
- At least two demo payment pages configured
- At least one completed test transaction in the system

### Core Features

- Payment page creation and configuration (branding, amounts, fields)
- Public payment page renders correctly and accepts payment

- URL copy, iframe snippet, and QR code generation
- Confirmation email sent on successful payment
- Reporting section with transaction list and summary

## Quality & Compliance

- Payment page passes Lighthouse accessibility audit (target: 90+)
- Keyboard navigation works end-to-end on the payment page
- No hardcoded credentials in source code
- README with setup instructions and architecture notes

## Submission

- GitHub/GitLab repository shared with organizers
- Hosted URL submitted via the hackathon portal
- Demo login credentials provided
- Team is prepared for 5-10 minute live demo

## Good luck, and happy building!

*Questions? Reach out to your hackathon coordinator.*